This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCAD.2018.2846625, IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems

ACHILLES: Accuracy-Aware High-Level Synthesis Considering Online Quality Management

Shayan Tabatabaei-Nikkhah[1], Mahdi Zahedi[1], Mehdi Kamal[1,2], Ali Afzali-Kusha[1], Massoud Pedram[3]
[1]School of Electrical and Computer Engineering, University of Tehran, Iran
[2]School of Computer Science, Institute for Research in Fundamental Sciences (IPM), Iran
[3]Department of Electrical Engineering, University of Southern California, USA
{shayan.tabatabaei, mahdi.zahedi, mehdikamal, afzali}@ut.ac.ir, pedram@usc.edu

***Abstract*—In this article, we present an accuracy-aware design framework (called Achilles), which synthesizes a high-level description of an input application with the objective of minimizing the energy consumption of the synthesized circuit. The proposed framework includes two main parts of accuracy-aware high-level synthesis and light-weight predictor selection. The framework leverages light-weight error predictors (i.e., machine learning-based classifiers) to achieve more energy reduction by dynamically managing the output quality level (exact or approximate) of the synthesized circuit. To synthesize the input application, first, we exploit a heuristic algorithm to determine the quality level required for each operation in the data flow graph (DFG) representation of the input application. Next, for synthesizing the input application, we propose an effective accuracy-aware high-level synthesis algorithm which utilizes the flexibility of the available multi-quality arithmetic units in a high-level cell library to synthesize the datapath. To improve the efficiency, the process starts by iteratively reducing the number of functional units required for synthesizing the data flow graph. Then, a proper light-weight error predictor satisfying the user expected quality is chosen from the available predictors in the framework. Based on the quality requirements, three different quality management modes are considered. The efficacy of the proposed framework is assessed for benchmarks from image and signal processing as well as robotics domains. The study of these benchmarks indicates that Achilles may reduce the energy consumption up to 51% (36% on average), up to 72% (51% on average), and up to 57% (33% on average) in Threshold, Average, and Hybrid modes, respectively for the studied cases. Moreover, the results show that relative coverage of large errors may be increased from 21% to 55% by employing SMOTE method.**

***Index Terms*—High-level synthesis, quality management, approximate computing.**

I. Introduction

Entering the post-Dennard era has led to a special emphasis on energy efficiency in the design of the state-of-the-art computing platforms. At the same time, there are large classes of applications in the embedded domain (such as multimedia and signal processing, machine learning, and robotics) which have an intrinsic resilience to imprecise computations. This fact has opened the opportunity for reducing the power consumption in this domain by using the approximate computing paradigm. In this paradigm, a trade-off between quality and energy consumption exists. Note also that in embedded systems which make use of accelerators, the approximate computing paradigm may be used to perform tasks faster with lower energies.

There are various types of approximate accelerators in the literature. Among them, neural accelerators and those implemented with approximate arithmetic units are the most noteworthy ones. Neural accelerators have been widely used in recently proposed architectures (*e.g.*, [1]–[2]). For these accelerators, when the number of neurons increases, the power becomes large, making them unsuitable for embedded targets. In general purpose high-performance computing systems, a high level of energy consumption by these accelerators may be acceptable.

Approximate accelerators, which tend to have significantly lower power consumptions than the precise ones, are more appropriate for embedded systems. Many approximate arithmetic units, including adders and multipliers, which are used to realize these approximate accelerators, have been proposed in the literature in recent years (see, *e.g.*, [3]–[7]). Reconfigurable (multi-quality) approximate arithmetic units with two selectable output quality levels of exact and approximate (see, *e.g.*, [5]–[7]) have also been proposed. These units enable the computation in both approximate and exact modes.

The steps of scheduling, allocation, and binding of operations to arithmetic-logic units in a datapath and that of designing a controller for the said datapath can be automated through a high-level synthesis (HLS). The HLS comprises several steps, including algorithm optimization, dataflow analysis, library processing, resource allocation, scheduling, functional unit binding, and register binding where among them scheduling and functional unit binding are the most significant ones. Traditional scheduling methods (such as list or force-directed scheduling algorithms) lack the ability to take advantage of the energy reduction opportunities available in accuracy-configurable arithmetic units. In [8], the conventional list scheduling algorithm has been improved by employing the fact that low precision operations can be bound to high precision arithmetic units. One can reasonably expect to achieve further improvements by employing the aforementioned reconfigurable arithmetic units.

It should be noted that the frequency and magnitude of output errors in a design utilizing approximate units may violate the minimum quality constraints. The intensity of the errors is in turn related to the applied input patterns. Classifiers, which are common tools in machine learning, may be used to categorize the data into several classes. They also may be employed to distinguish between inputs that tend to result in large errors and those "benign" inputs that do not cause much errors (in

0278-0070 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCAD.2018.2846625, IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems

magnitude) [1][2][9]. When accelerator invocations whose inputs lead to large output errors are identified, quality violations can be avoided by re-computing the results for these inputs precisely. The recomputation commands are issued by the classifier (error predictor) which is also synthesized along with the accelerator. For a given quality constraint, the error predictor, which is trained offline, issues rollback commands in the online phase when the inputs are more likely to cause unacceptably large output errors for that quality level.

In this work, we propose an ACcuracy-aware HIgh-LEvel Synthesis framework (named Achilles), which consists of an approximate accelerator (synthesized using approximate arithmetic units) and classifiers which are used to predict approximation-induced errors. To automatically synthesize the accelerator based on the behavioral code, a scheduling and binding methodology which improves the previous HLS techniques in terms of energy consumption is presented. The methodology makes use of the accuracy reconfigurability of the recently proposed approximate arithmetic units for reducing the number of functional units in the synthesized datapath. Furthermore, to attain a more efficient resource sharing scheme, approximate operations are bound to more precise functional units wherever possible. By doing so, considering the quality constraints, the proposed framework can provide us with a better quality (quality slack) than the required level. This can be utilized to increase the approximation level of the accelerator for achieving more energy savings. To improve the accuracy of classifiers, two techniques of ensemble learning and SMOTE (Synthetic Minority Oversampling Technique) are utilized. To do a detailed evaluation, we compare the classifiers using ROC (Receiver Operating Characteristic) plots and select the most appropriate one. The contributions of this work are summarized below:

- Present an accuracy-aware design framework for performing high-level synthesis when considering online output quality management,
- Describe an accuracy-aware HLS algorithm which makes use of multi-quality approximate arithmetic units, and
- Improve the effectiveness of the error predictors by under- and over-sampling technique during the training phase.

It should be mentioned that none of the prior HLS works has employed any of the techniqeues listed in the above contribution list.

The rest of the article is organized as follows. Section II reviews some of the works performed in the contexts of approximate computing, high-level synthesis, and quality management. In Section III, first an overview of the proposed framework is presented and then the design of the accelerator and the error predictors considering different quality management modes are described. The efficiency of the proposed method is evaluated in Section IV. Finally, the article is concluded in the last section.

II. Related Works

There are quite a number of works in the area of approximate computing. Some of the proposed approaches seek energy gains in the software layer, while others have chosen the hardware layer. Among hardware-layer approaches, precision scaling (see, *e.g.*, [10]–[11]), voltage overscaling (see, *e.g.*, [10]), and employing inexact logic (see, *e.g.*, [12]) are the most popular ones. In this work, the approximate computing based on imprecise logic approach is employed for realizing an approximate accelerator operated along with a host.

Few works dealing with automating the process of approximate circuit synthesis have been reported in the literature. A two-phase approach, which tries to reduce the number of literals in a Boolean network for given error rate and error magnitude constraints, has been proposed in [13]. SALSA [14], which uses traditional *don't care* based optimization techniques to simplify circuits, has utilized quality constrained circuits. By extending SALSA, another framework called ASLAN, which synthesizes approximate sequential circuits with the objective of lowering the energy consumption for a given quality constraint, has been proposed [15]. In the high-level synthesis domain, a tool called ABACUS [16] has been proposed. To synthesize the optimal design, it applies operator transformations to an abstract synthesis tree extracted from behavioral description of the input application. The scheduling and binding, as the essential parts of the HLS, however, were not discussed in the work. Moreover, a method of finding an optimal design configuration based on Integer Linear Programming (ILP) has been suggested in [8] where a simple linear model for the error estimation was proposed as well. Additionally, in this work, the conventional list scheduling algorithm is improved by iteratively reducing the number of functional units in the datapath. When the linear equations employed for the error estimation are not sufficiently accurate (*e.g.*, in the case of data flow graphs containing feedback loops), simulations or semi-analytical techniques may be needed. Consequently, finding the optimal design configuration using this method would not be always possible. Additionally, the method may not take advantage of more reductions of the functional units by using multi-quality arithmetic units.

Several works have dealt with managing the output quality. For example, Green [17] and SAGE [18] assess the output quality once in every N invocations and adjust the aggressiveness of the approximation based on the current quality and user requirements. However, between two subsequent quality checks, large errors may exist in the outputs, which can be unacceptable for the user. Therefore, rollback recoveries are needed to recompute the outputs with large errors. To address this issue, the three frameworks discussed in [1], [9], and [2], which equip their approximate accelerators with error predictors, have been proposed. For each application iteration, these error predictors decide whether a rollback recovery is needed or not. This is performed by predicting the output error based on the inputs and comparing it with a given error threshold. Three basic predictors have been proposed in Rumba [1], namely, the linear predictor, the decision tree, and the moving average based predictor. In addition to introducing a table based predictor, [9] and [2] have used ensembles of classifiers to improve the accuracy. Given the fact that the error predictors must be light-weight in nature, the number of

0278-0070 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCAD.2018.2846625, IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems

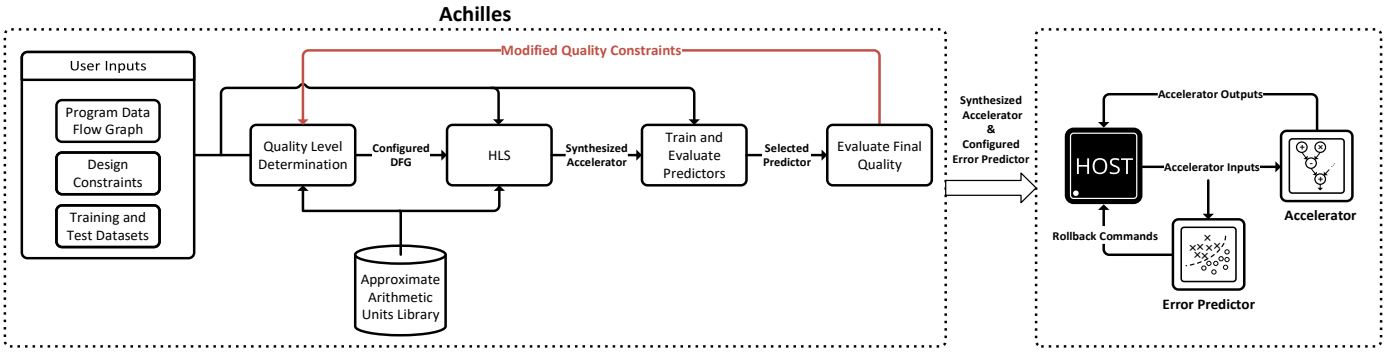


Fig. 1. The proposed framework (Achilles).

classifiers which are used in an ensemble will be limited. Considering this limit, we use oversampling and undersampling to enhance the efficiency of the prediction without further hardware overhead. To perform a comparison between the error predictors for choosing the better one for an application, we compare them using ROC plots [19].

III. Proposed Design Framework (Achilles)

A. Framework Overview

The general flow of the proposed design framework of Achilles is given in Fig. 1. The inputs of this framework, which are provided by the user, include data flow graph (DFG) extracted from the part of the program which must be executed by an accelerator (*i.e.*, hot-spot part), quality constraints (including average quality and error threshold), delay constraints, and data samples used to train and evaluate the error predictors. In addition, a library of approximate arithmetic units should be provided. The output of this flow will be an accelerator along with a light-weight predictor which dynamically manages the output quality of the accelerator during the runtime. Whenever the error predictor predicts a quality violation, it sends a rollback command to the host. In this work, since we suggest using reconfigurable approximate arithmetic units, the designed accelerator (in fully exact operating mode) is used to execute the rollback recoveries.

In the flow, first, the impact of the quality level of each operation on the accelerator output quality is determined. The proposed heuristic algorithm explores different configurations for the circuit (based on the impact of the operations on the accelerator output quality and their energy savings) without violating the accelerator expected output quality. The value of the quality constraint for this phase is determined by the quality management part. Next, the high-level synthesis (scheduling and binding) is performed based on the design parameters of the arithmetic units available in the library as well as the defined constraints.

In the proposed design approach, the output of the synthesized accelerator may violate the expected quality (error threshold) in some cases. Hence, after synthesizing the accelerator, a predictor unit, which predicts the input cases that may result in violating the output quality constraint for the circuit, is generated. Here, for selecting the better predictor unit from two well-known classifiers, we describe an exploration approach based on the ROC plots. Next, as the configuration of the accelerator and the predictor are specified, the application quality resulting from the operation of the accelerator along with the error predictor are assessed to determine the satisfaction of the quality requirement. If quality violations occur, based on the quality management mode, the accelerator output quality constraint (the average quality constraint employed to configure and synthesize the accelerator) and the error threshold (the constraint set for training of the predictor) are updated, and the design flow is repeated until the user quality constraints are all met.

B. Hardware Accelerator Design

One of the most important and most challenging issues in the approximate high-level synthesis is to determine the proper accuracy level of the components of the circuit (accuracy configuration). The levels should be selected with the objective of minimizing the energy (maximizing the energy saving) for a given output quality constraint. In this section, we provide an HLS technique which includes both scheduling and binding to solve this problem. The proposed technique automatically determines the circuit accuracy configuration. In the following, first, we discuss the algorithm used in *Quality Level Determination* part which determines the output quality level of each module in the circuit. Then, the proposed algorithm for scheduling and binding (*HLS* step) will be discussed.

1) Quality Level Determination

To find the optimal accuracy configuration of the circuit, an exhaustive search, in which all possible configurations are simulated, may be used. The number of possible combinations for N components that have M levels of quality operations is M^N. Fig. 2 shows the exhaustive simulation runtimes for different benchmarks (the size of the benchmarks are reported in Section IV) under two different quality levels. The reported run times indicate that by increasing the number of components and their levels of quality, the exhaustive simulation becomes time-consuming and impractical. Based on this observation, we propose an exploration approach which is adapted from the method proposed in [15]. The approach, which finds a sub-optimal configuration in a considerably shorter time, is applied to approximate circuits that include multi-quality level modules (see the description below). The main inputs to the algorithm are the data flow graph and the quality constraints provided based on the quality management mode of the system (see Subsection 3.D).

The arithmetic components of the circuits, such as adders and

0278-0070 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCAD.2018.2846625, IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems

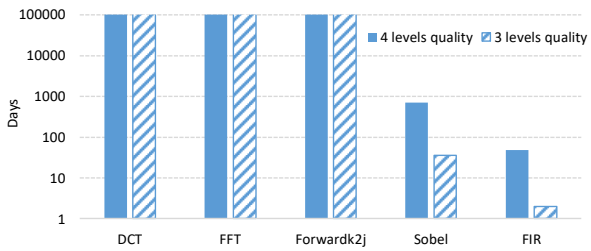


Fig. 2. Run times of exhaustive simulations for finding the best configurations.

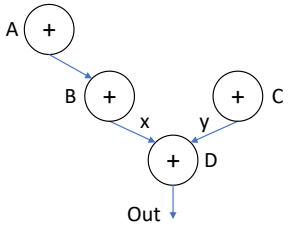


Fig. 3. An example of a task graph.

multipliers, are good candidates for operating in the approximation mode. For properly-designed approximate units, the error amount is directly related to the gained power. To maximize the energy reduction using the approximate approach, approximate units whose errors have smaller impacts on the output quality (and result in higher energy reductions in the approximate mode) should be selected. Based on this, in the proposed exploration approach, we define *merit value* of the ith component in the jth quality level ($M_{(i,j)}$) as

$$M_{(i,j)} = \frac{E_{(i,E)} - E_{(i,j)}}{\mathrm{MRED}_{(i,j)}} \tag{1}$$

where $E_{(i,E)}$ ($E_{(i,j)}$) is the energy consumption of the ith component in the exact (j^{th} quality level) operating mode while all the other components are in the accurate mode. Also, $\mathrm{MRED}_{(i,j)}$ is the Mean Relative Error Distance of the DFG output, when the ith component is in the jth quality level while the other components are in the accurate mode. In the proposed merit function, for each component, in addition to its power consumption, its accuracy impact on the output quality when other components are operating in their exact modes is considered. This way of calculating the merit function value of an exact vs. approximate calculation, leading to a shorter run time, is based on the assumption that the impact of an imprecise module on the output is approximately independent of the quality levels of the other modules [15]. In other words, the error generated due to each sole approximate component at the output provides us with a sensible clue about the error amount portion of that component imposing on the output quality while operating in the approximate mode. As an example, for the graph shown in Fig. 3, when the resource B operates in the approximate mode, the Mean Relative Error Distance (MRED) of the graph output is expressed by

$$\begin{aligned}\mathrm{MRED}_{\{B_{apx}\}} &= \frac{1}{N}\sum_{i=1}^{N}\frac{\left|(x_i + y_i) - (x_i + y_i + e_{B_i})\right|}{x_i + y_i} \\ &= \frac{1}{N}\sum_{i=1}^{N}\frac{\left|e_{B_i}\right|}{x_i + y_i}\end{aligned} \tag{2}$$

where $\mathrm{MRED}_{\{B_{apx}\}}$ denotes the MRED of the output when B operates in the approximate mode, N is the number of the inputs in the dataset, x_i and y_i are the exact values for the ith iteration, and e_{B_i} is the error generated by B at its output for the ith input set. In the case when both resources B and C are in the approximate mode, the MRED of the output is obtained by (3) which shows the first-order justification of the independency between the generated errors by the two approximate components. Note that (3) may not be true in every case and we employ it only for this example.

$$\begin{aligned}\mathrm{MRED}_{app=\{B,C\}} &= \frac{1}{N}\sum_{i=1}^{N}\frac{\left|(x_i + y_i) - (x_i + y_i + e_{B_i} + e_{C_i})\right|}{x_i + y_i} \\ &= \frac{1}{N}\sum_{i=1}^{N}\frac{\left|e_{B_i}\right|}{x_i + y_i} + \frac{1}{N}\sum_{i=1}^{N}\frac{\left|e_{C_i}\right|}{x_i + y_i}\end{aligned} \tag{3}$$

It is worth noting that the error generated by an approximate module depends strongly on its inputs. Although other approximate modules cause changes to these inputs, our experiments for FIR and FFT benchmarks show that the order of nodes in the sorted list does not considerably change, implying that the determined configuration based on this method yield results similar to those of the optimum configuration (with up to 15% increase in energy consumption compared to the optimum configuration).

In the proposed heuristic algorithm, possible quality levels for all the nodes in our DFG are sorted based on their merit values. As an example, for the DFG shown in Fig. 3, assuming that we have three quality levels for each node, our list contains twelve elements sorted based on their merit values. In each iteration, first, the accuracy level of a node with the highest merit value is selected and applied to the circuit. If this does not cause any quality constraint violation, the selection is approved. Note that quality evaluations are performed using simulations. This process is repeated for all the elements on the sorted list. It is worth noting that since we evaluate the quality in each iteration, the quality constraint will always be met. The time complexity of our proposed algorithm is $O(n \times m)$ where n is the number of modules and m is the number of approximate levels, while the time complexity of the exhaustive search is $O(m^n)$.

2) Accuracy-Aware High-Level Synthesis

The list scheduling algorithm, which is one of the well-known scheduling methods in the literature, is able to either minimize the resource usage subject to a latency constraint, or minimize the latency subject to a resource constraint. However, employing approximate functional units provides additional opportunities to schedule the circuit for energy reduction. The work of [8] deals with two problems of the list algorithm for the approximate applications and presents a method for solving them. The first problem is that sometimes one may prefer to bind a low precision operation to a high precision resource (but not vice versa) [8] so that more resources can be shared, resulting in a decrease in the static power. The second problem is the initial number of resources for starting the algorithm in the latency constraint mode. This number can affect the total

0278-0070 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCAD.2018.2846625, IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems

Algorithm 1 *Proposed accuracy-aware scheduling algorithm.*

```
1:  function ACHILESS Scheduling (G, Quality_constraint, λ̄)
2:      IR =[1]m×n ;
3:      i = 0;
4:      while E_i < E_{i−1} do
5:          (R,t) ← LIST_R(G, λ̄,Resource = IR);
6:          if (t == 0 ) then
7:              return (0);
8:          end if
9:          for each exact operation node : ω do
10:             Candidates ← Idle resources which have type
                            τ(ω) in cycle(ω);
11:             R(ω) ← Resource that has maximum
                       utilization in circuit;
12:         end for
13:         for each resource ∈ R do
14:             τ ← Operation type of the resource;
15:             L ← Approximate level of the resource;
16:             Ins ← Instance number of the resource;
17:             Ready_List ← Nodes using resource "τ", level of
                "L", and instance number of "Ins";
18:             (L′, Ins′) ← Quality_Conversion (τ_{L,Ins}, R, λ̄)
19:             if L′ ≠ 0 then
20:                 for each node ∈ Ready_List : ω do
21:                     R(ω) ← τ_{L′,Ins′} ;
22:                 end for
23:             end if
24:         end for
25:     i = i + 1 , Increment(IR,1);
26:     end while
27:     Candidate ← Sort_Configs (G,R); // Candidate is
                                  the set of (τ_L, ω)
28:     for j=1 : size of Candidate do
29:         (ω, τ_L) ← Candidate_j
30:         if (There is vacant resource of τ_L in t_ω) then
31:             Ins ← Instance number of the resource
32:             Change resource of ω to τ_L;
33:             Q ← Evaluate quality; // any method
34:             if (Q > Quality_constraint) then
35:                 R(ω) ← τ_{L,Ins} ; // binding
36:             end if
37:         end if
38:     end for
39:     return (R);
40: end function
```

number of resources and, hence, the total energy consumption. This challenge has been addressed by proposing an iterative approach, instead of one-pass graph traversal, for the scheduling algorithm [8]. In this work, we improve the algorithm suggested in [8] in two ways. First, by employing reconfigurable arithmetic units, our proposed approach binds exact operations to reconfigurable approximate functional units (which could be configured for the exact and approximate mode), resulting in more resource sharing and fewer functional units. Second, our approach utilizes the quality slack (explained next) to increase the accelerator aggressiveness leading to more energy savings.

In the *Quality Level Determination* step, explained in the previous subsection, an appropriate configuration for the approximate circuit is determined. In that step, the accuracy level for each node is specified and used by the HLS algorithm

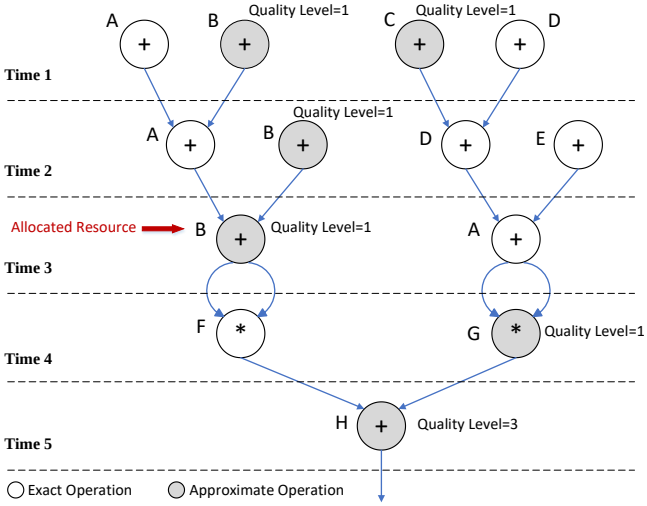


Fig. 4. Configuration of arithmetic units in Sobel benchmark under 30% quality loss.

for binding each node to a unit. The overall pseudo code of the proposed HLS (scheduling and binding) algorithm whose objective is to minimize the energy consumption subject to the latency and accuracy constraints is provided in Algorithm 1, where G and $\bar{\lambda}$ denote data flow graph and latency constraint respectively.

As previously noted, the number of resources which are available in the beginning of the algorithm is important and may affect the total number of resources as well as the total energy consumption. To tackle this challenge, in a greedy approach, our proposed algorithm iteratively increases the number of initial resources. This iterative procedure continues as long as increasing the initial resources has a positive impact on the total energy consumption (line 4). In Algorithm 1, the parameters E_i and IR (lines 4 and 2) denote the energy consumption including static and dynamic energy consumption which is consumed in iteration "i" and the number of Initial Resources for each type (*e.g.*, adder and multiplier) and quality level (m types and n quality levels in total) respectively. To clarify, $IR+i$ means we count all elements of IR vector by "i". E_{-1} and E_0 are two large positive numbers which are initialized in such a way that loop condition for the first iteration becomes true.

In the proposed algorithm, at the beginning of each iteration (line 5), the conventional list algorithm is employed as an initial scheduling approach. Note that any other scheduling algorithm can be used to do the initial scheduling, and the proposed approach is not confined to the list algorithm. The list algorithm returns tuple of (R,t) which shows that the resource R should be operated in cycle t. In the case when the start time is zero, the list scheduling could not schedule the DFG and the algorithm will be terminated (lines 6-8).

To perform arithmetic operations in the approximate circuit, we exploit recently proposed reconfigurable approximate arithmetic units which are able to switch between exact and approximate modes online. This feature may lead to reducing the required number of resources and static energy consumption. This is demonstrated by the example given in Fig. 4 where the latency deadline is five cycles and both the adders and the multipliers have the delay of one cycle.

0278-0070 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCAD.2018.2846625, IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems

Algorithm 2 *Replacement of resources with more accurate ones.*

```
1:  function QUALITY_CONVERSION (τ_{L,Ins}, R, λ̄)
2:      Clks ← Time steps which τ_{L,Ins} is used;
3:      for L' = L − 1: 1 do
4:          if (τ_{L'} ∈ R & (∃ idle instance of τ_{L'} in CLks: Ins')) Then
5:              clk_τ_L ← Number of clock that τ_{L,Ins} used;
6:              if leak_{τ(L)} * λ̄ > (pow_{τ(L')} − pow_{τ(L)}) * clk_τ_L then
7:                  return (L', Ins');
8:              else if L'==1 then
9:                  return (0);
10:             end if
11:         end if
12:     end for
13: end function
```

The reconfigurable arithmetic units can also operate in the exact mode, and hence, exact operations may be bound to them. Let us consider the second cycle where we need three functional units to perform exact Add operations. In this cycle, the resource *C* with the approximate level 1 is idle. If this resource is a reconfigurable approximate adder, one of the three exact operations could be bound to it reducing the total number of resources by one. The functional resource *C* operates in the exact mode in the second cycle while it operated in the approximate mode in first cycle (not indicated in the figure). If there are several choices for binding an operation, we choose the resource with the maximum utilization to increase the chance of reducing the number of the required functional units (see lines 9 to 12 in Algorithm 1). Thus, for each of the exact operation nodes in the DFG, candidate resources which have the same type with that node and are idle in its time step are determined where, finally, the one with the maximum utilization in the circuit is bound to that node.

As demonstrated in Fig. 4, the resource *H* is just used in one time step while being idle (and dissipating energy due to the leakage) in other time steps. To determine if a switching of the quality level of a functional unit is advantageous in terms of energy consumption, the function *Quality_Conversion* is employed in our algorithm (line 13-24). More specifically, for each approximate node, we determine whether its binding to a more precise functional unit is beneficial in terms of the total circuit energy or not. A saving in the total energy occurs when the rebinding to a more accurate unit causes the elimination of a resource. For this case, there should be a free instance of the target resource (functional unit with higher precision) in all time steps. As an example, consider the circuit shown in Fig. 4. In time step 5, the DFG requires an approximate adder with the quality level of 3. We have two approximate adder instances with the quality level of 1, which are idle in Time 5 (when the resource *H* is used) and hence, one of them may replace *H*. This leads to leakage current reduction. The energy gain due to the replacement (rebinding) should be obtained for all the candidates. The replacement takes place if the static energy reduction is higher than dynamic energy increase.

The pseudo-code of *Quality_Conversion* algorithm is shown in Algorithm 2. In this algorithm, $\tau_{L,Ins}$ is the resource with the operation type of "τ", approximate level of "L", and instance number of "*Ins*" which is replaced by another resource whenever possible. Also, *R* is the set of all resources which are used in the circuit and $\bar{\lambda}$ is the latency of the circuit. Additionally, *pow* is energy per clock cycle and each resource has its own value.

Algorithm 3 *Algorithm for sorting different configurations.*

```
1:  function SORT_CONFIGS (G, R)
2:      for each node in the circuit : ω do
3:          τ_L ← R(ω)
4:          for (All approximate levels which are larger than L: L') do
5:              if (number of resources reduced) then
6:                  Merit (ω,τ_{L'}) ← (pow|_{R(ω)=τ(L')} − pow|_{R(ω)=τ(L)}) + Leakage_power (τ_L) / (Quality|_{R(ω)=τ(L)} − Quality|_{R(ω)=τ(L')})
7:              else
8:                  Merit (ω,τ_{L'}) ← (pow|_{R(ω)=τ(L')} − pow|_{R(ω)=τ(L)}) / (Quality|_{R(ω)=τ(L)} − Quality|_{R(ω)=τ(L')})
9:              end if
10:         end for
11:     end for
        Candidate ← Sort tuples in descending order of their merit values;
12      return (Candidate);
13: end for
```

If the algorithm can replace the resource, the final quality may become greater than before. One may take advantage of this quality improvement and replace some of functional units in the synthesized datapath with those with less accuracy to further reduce the energy consumption. Thus, in the last step of the proposed HLS algorithm, an exploration is performed for relaxing the quality levels (increasing the inaccuracy level) of the nodes which were determined before. For this purpose, the *Sort_Configs* function, whose pseudo-code is shown in Algorithm 3, is suggested. In this function, all of the feasible node relaxations are explored and the one(s) providing the highest reduction in the total energy reduction is (are) selected. In this algorithm, for all the nodes in the circuit (line 2), the proper merit values for the resources which are less accurate than the current resources to which the nodes are bound (line 4) are calculated. Obviously, it is more profitable if the rebinding can reduce the number of resources due to leakage power and area usage reduction. This corresponds to the first merit expression (line 6). In the case when the number of resources is not reduced, we suggest to employ another merit function (line 8) considering only the dynamic energy improvement which may provide us with a greater energy gain and/or lower quality loss. The merit values for all possible rebindings should be obtained in the *Sort_Configs* function. The rebinding merit values are sorted and placed in the *Candidate* list (line 12) which contains tuples of (ω, τ_L) indicating the resource τ_L to which the node ω should be bound.

In the proposed accuracy-aware HLS algorithm, by considering the candidate list, a node will be rebound if its alternative resource is idle in the time step of the selected node execution. As the next step, the final quality after this rebinding is evaluated. In the case where the quality constraint violation occurs, the rebinding is undone and the next candidate for replacing the node is considered and the process continues until no constraint violation occurs or we reach to the end of the list

0278-0070 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCAD.2018.2846625, IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems

(line 28-38 of Algorithm 1).

C. Error Predictors

As discussed before, there might be some accelerator output values with unacceptable approximation errors. The accuracy of these outputs should be improved before finalizing the output results (values). For this purpose, we employ error predictors to speculate whether the output values have errors exceeding a desired error threshold or not. If all the accelerator outputs have errors below the error threshold, the error predictors will not be used. For the error prediction, we suggest utilizing machine learning classifiers. The classifiers may be used to either predict the value of the error or determine if a rollback operation (exact computation) is needed by putting a binary label (*e.g.*, $\{0,1\}$) on the accelerator output value. In the former case, the predicted error is compared to a specific threshold value to determine if the rollback operation is required. As suggested in [1] and [9], the inputs of the error predictors can be chosen from the accelerator inputs, intermediate data, and outputs. Considering the fact that the delay of the prediction should not be larger than the accelerator critical path, we exploit the accelerator inputs as to train the classifiers. In this section, first, we introduce the basic classifiers which may be employed as the error predictors, and then we present some methods for improving them. Furthermore, a technique for evaluating their performances and comparing them will be presented.

1) Basic Classifiers

There are quite a few algorithms in the machine learning literature to address the classification problem (see, *e.g.*, [20]). Among them, those with simpler hardware which have less power consumption and smaller critical path delay are more appropriate for our framework. Thus, we selected linear classifiers and decision trees as the basic classifiers where ensemble methods were employed to increase their classification capabilities without increasing the hardware complexity significantly [9]. Furthermore, here, since we are facing a class-imbalanced classification problem [31], *SMOTE* technique which improves the classification accuracy by changing the distribution of samples in the training dataset is employed [21]. The two classifiers are elaborated next.

Linear classifier: A linear classifier, first, estimates the output error by combining the features (accelerator inputs in our case) and, then, compares the estimated error with a given error threshold. When the estimated error is greater than the error threshold, the classifier output becomes one, indicating the need for the exact computation by the accelerator. The linear model estimates the error as [1]

$$\hat{e} = \omega_0 \times x_0 + \omega_1 \times x_1 + \cdots + \omega_n \times x_n + c \qquad (4)$$

where ω_i is the weight of the ith input (denoted by x_i) and c is a constant. All the parameters (ω_is and c) are computed offline.

The hardware for the linear classifier is mainly comprised of a MAC unit (which iteratively computes the error) and a comparator (to compare the estimated error with the precomputed error threshold) which are light-weight enough not to impair the advantages gained through using the approximate accelerator.

Decision tree: A decision tree is a flow-chart like tree structure consisting of internal and leaf nodes. In each internal node, a decision of whether to traverse the left child or traverse the right one is made. In our case, this decision is made through a comparison between one of the accelerator inputs and a threshold computed during the training phase by employing the training data set. Each leaf node holds a class label which in our case is one, representing errors beyond the threshold, or zero (for non-destructive errors). To avoid an excessive tree power consumption, as suggested in [1], we restrict the depth of the trees. The light-weight hardware needed for implementing this classifier is a comparator accompanied by a simple controller to provide the proper data in each iteration. In order to determine the depth limit of descision tree for each benchmark, we first consider an energy consumption upper bound for the error predictor ($E_{predictor}$) based on the energy consumption of different parts of the system ($E_{accelerator}$, $E_{rollback}$, and $E_{predictor}$) and expected energy gain of approximation (α) using (5):

$$E_{accelerator} + R \times E_{rollback} + E_{predictor} \leq \alpha \times E_{exact} \qquad (5)$$

where R is the rate of the rollback recoveries. The rate may be estimated using the training dataset and the error threshold. Then, based on this upper bound and energy consumption of the elements in decision tree circuit, the depth limit is determined.

2) Ensemble Methods

Ensemble methods are learning algorithms which exploit multiple classification models to improve the overall performance (accuracy) by taking a weighted vote of their predictions. A comprehensive study comparing the ensemble methods have been performed in [22]. Based on this study, the AdaBoost (Adaptive Boosting) which often worked best among other ensemble methods was chosen for our work.

AdaBoost: The boosting [23] is achieved by producing a series of "weak" classifiers (which have accuracies only slightly better than pure chance) in order to generate a powerful combination. To do so, in each iteration of this iterative algorithm, the distribution of the training set changes based on the performance of the previous classifiers. This reweighting of the samples is performed in such a way that misclassified samples gain more weight, resulting in a greater emphasis on misclassified samples and a better prediction accuracy for the next classifiers. The final classification is accomplished through a voting procedure.

The algorithm input is a training set $(x_1, y_1), \ldots, (x_m, y_m)$ where each x_i is a vector with n elements (in our case, n is the number of accelerator inputs) and each label y_i is selected from a label set Y which usually equals to $\{-1, +1\}$. Initially, the weights of the input tuples are set equally, that is $1/m$ for m tuples; however, in each round, the weights of the misclassified samples increase, resulting in a greater focus on them in the next round. On the round t, the parameter α_t (which is used to change the weights of the samples and basic classifiers) is computed using the error of the most recently created classifier. Finally, the classifications of the T basic classifiers are combined through a weighted majority vote to produce the final classification:

0278-0070 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCAD.2018.2846625, IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems

$$H = sign(\sum_{t=1}^{T} \alpha_t h_t) \quad (6)$$

where h_t is the predicted label of the classifier created on the tth round, and H is the predicted value resulted from the majority voting.

The hardware implementation of AdaBoost includes T basic classifier circuits and a weighted majority voting circuit. The number of basic classifiers is limited according to the power consumption constraint of the application.

3) Improving the Accuracy of the Classifiers

When the number of samples belonging to the main class of interest (inputs resulting in large errors in our case) are significantly lower than the number of other class tuples, we have a class imbalance problem [21]. It has been demonstrated in [1] that using an approximate accelerator leads to large errors in less than 20% of outputs. In other words, less than 20% of the samples belong to the large error class, resulting in a class-imbalanced problem. Almost similar case is valid for our proposed framework.

Standard machine learning algorithms choose models with fewer misclassified samples. These models have the tendency of labeling all samples as the majority class leading to the misclassification of the minority class (main class of interest). To increase the sensitivity of classifiers to the main class of interest, we equalize the number of tuples in each class. To do so, we can either under-sample tuples of the majority class or over-sample tuples of the minority class. An over-sampling technique (named SMOTE) was proposed in [21]. In this technique, the minority class is over-sampled by creating synthetic samples considering each sample and its neighbors. For each sample of the minority class, the differences between its feature vector and vectors of its k nearest neighbors, depending on the percentage of over-sampling, are multiplied by random numbers between 0 and 1. By adding these difference vectors to the sample under consideration, synthetic samples are generated. These synthetic samples cause a wider and more general decision regions for the minority class.

In [21], it has been shown that combining the aforementioned approaches of under-sampling and over-sampling led to a better classification accuracy than applying over-sampling and under-sampling separately and, hence, this approach was used in our work. In addition, in our work, the energy consumption of the classifiers is lowered by feature selection [20]. In our classification problem, there might be some features (accelerator inputs in our case) that may be eliminated without affecting the classification accuracy significantly. The elimination makes the prediction process faster with consuming less energy. There are a number of algorithms proposed in the literature for finding the best feature subset. These algorithms are useful when an exhaustive search requires unacceptably long runtime. For the applications that we considered in this work (and other common applications used in approximate computing literature), the exhaustive approach may be used.

4) Model Evaluation

Having built the classification model, the performance should be evaluated for the unseen data. The most common evaluation metric, among different evaluation metrics in the literature, is accuracy (also known as recognition rate) (see, *e.g.*, [24]). When the datasets are class-imbalanced, similar to our case (see [1]), and also when the cost of misclassifications are not identical (quality loss versus rollback energy overhead loss) the accuracy may not be a proper metric. Here, to assess how well our classifiers can recognize both classes, above and below the error threshold, we employ **sensitivity** and **specificity** metrics. The sensitivity parameter, also referred to as the true positive rate, shows the percentage of tuples correctly labeled as positive. The specificity parameter, also known as the true negative rate, shows the percentage of negative tuples which are properly classified. In the next subsection, we exploit these two metrics simultaneously to compare the classifiers by a diagram called ROC (Receiver Operating Characteristic) plot.

Another important issue, related to the reliability evaluation, is that an estimation based on training datasets is overly optimistic and, hence, it should be avoided. There are plenty of techniques to address this issue where holdout and random subsampling, cross-validation, and bootstrap methods are the most common ones [24]. The extensive studies performed in [25]–[27] indicate that the stratified k-fold cross-validation (especially, for $k = 10$) is recommended in general. In this work, we employed the stratified 10-fold cross-validation.

5) Model Selection

Having several classifiers with different sensitivities and specificities, one should select the best one. For the selection purpose, each classifier is displayed in the ROC space, and compared using the method introduced in [19]. The ROC curve plots FPR (False Positive Rate) which is the proportion of the negative samples mislabeled as positive (on the x-axis) and TPR (True Positive Rate) which is the rate of the positive samples correctly classified (on the y-axis). In other words, it shows the trade-off between TPR and FPR. Each classifier may be indicated by a point using previously estimated values as its coordinates.

The point that is closer to the top-left corner (point (0,1)) shows the classifier with the better performance in correctly classifying the samples of both classes. The line connecting the point (0,0) to (1,1) represents a random classifier, and therefore, classifiers whose representing points are below this line are worse than a random guesser. An example of an ROC graph is shown in Fig. 5 representing four different classifiers. In this example, the corresponding point of the classifier C in the ROC space is below the random guesser line. The convex hull has been suggested as a way to compare the classifiers and choose the best one for specific misclassification costs and class distributions [19]. We plot the convex hull of the classifier points along with (0,0) and (1,1), which are the extreme cases in the figure. The comparison is performed using iso-performance lines. These lines are sets of points where their corresponding classifiers have the same expected cost. A classifier, represented by a point (FPR, TPR), has an expected cost of [19]

$$Cost = P(pos) \times (1 - TPR) \times C(neg, pos) + P(neg) \times FPR \times C(pos, neg) \quad (7)$$

0278-0070 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCAD.2018.2846625, IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems

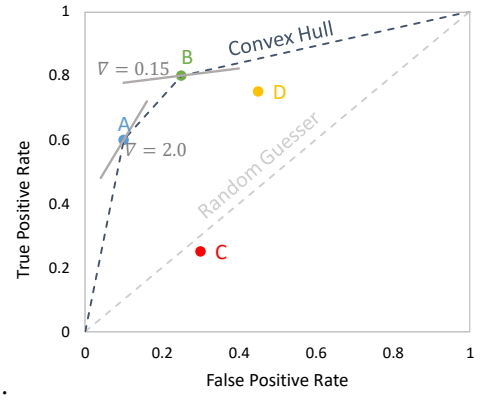


Fig. 5. An example of ROC plot.

where $P(.)$ is the prior probability of a class (related to class distributions) and $C(x, y)$ is the cost incurred if a class y sample is mislabeled as being in class x. If two classifiers have an identical expected cost, one can write

$$\frac{TPR_2 - TPR_1}{FPR_2 - FPR_1} = \frac{P(neg) \times C(pos, neg)}{P(pos) \times C(neg, pos)} \quad (8)$$

which corresponds to the slope of an iso-performance line (∇ in Fig. 5). For each set of misclassification costs and class distributions, a group of iso-performance lines which have an identical slope may be specified.

Each classifier which is located in the convex hull is potentially optimal for a range of class distributions and costs (a range of iso-performance line slopes). Thus, after drawing the convex hull, we first specify the misclassification costs. Then, considering the posterior probabilities of each class, the desired iso-performance line slope can be calculated using (8). The optimal classifier is the one that lies on the most northwest iso-performance line [19]. In other words, it lies on the iso-performance line which intercepts the TPR-axis at a higher point (compared to other classifiers on the convex hull).

In the example depicted in Fig. 5, the classifiers A and B are lying on the convex hull, which makes them better candidates for being selected as our classifier. If we define the misclassification costs ($C(pos, neg)$ and $C(neg, pos)$ in (8)) in such a way that the corresponding slope becomes 2.0, as shown in Fig. 5, the classifier A will be the best one. On the other hand, if the desired slope is 0.15, the classifier B will lie on the more northwest iso-performance line with the corresponding slope (compared to the classifier A) and, hence, it would be the chosen classifier.

D. Quality Management

The average and maximum of the errors are two quality metrics which have been considered in approximate computing [28]. To propose a comprehensive design framework, the three following quality management modes are considered in this work:

Threshold Mode: In this mode, the user provides an error threshold (maximum of error) and all the output values should have errors smaller than this error value. To avoid quality violations, the error predictors are trained with an error threshold less than or equal to the provided threshold. Obviously, the lower the threshold value is, the higher number of rollbacks may be required while the higher threshold value is, the lower the quality would be. For the former case, more approximation may be used in the accelerator to lower the energy overheads due to extra rollbacks (and even the total energy consumption). The challenge for this mode is to determine a better error threshold value and the accelerator average quality. This is performed using an iterative greedy approach with the objective of minimizing the total energy consumption. In each iteration, the error threshold (E_{Th}) is defined by a mechanism (discussed below), and then the proposed design framework is called with this threshold and the average quality (described next) as the quality constraints. At the end of the ith iteration, the error threshold for the $(i+1)^{th}$ iteration ($E_{Th_{(i+1)}}$) is determined from

$$E_{Th_{(i+1)}} = E_{Th_i} + \alpha \times \Delta E_{Th} \times \gamma \quad (9)$$

where α is scaling factor ($\alpha \in (0,1]$), ΔE_{Th} is a predefined step size, and γ is the direction factor ($\gamma \in \{-1,1\}$). The default value for γ is -1. At the end of each iteration, if the energy consumption is increased (compared with its value in the previous iteration), we change the sign of direction factor. In addition, when the energy consumption reduces, while it had increased in the previous iteration, the parameter α will be reduced by 20% (the initial value of α is 1). The iterations are terminated whenever α becomes smaller than a predefined value (say, 0.05). In addition, the initial value of the error threshold is equal to the threshold provided by the user.

Considering the fact that the average quality is not defined by the user, a similar procedure is used to determine the average quality in each iteration. The initial value of the average quality is defined as the half of the quality threshold.

Average Mode: In this mode, an average quality constraint for the target application is specified by the user. As mentioned before, the accelerator average quality and the error threshold for the predictor design should be determined. Here, the iterative technique proposed in the *Threshold Mode* is also used. For this mode, first, as the initial accelerator average quality, a value slightly lower (*e.g.*, 30% smaller) than the final average quality which is given by the user is specified. The initial value for the accelerator average quality was obtained empirically with the goal of minimizing the number of the required iterations needed for reaching the sub-optimal point. In each iteration, after synthesizing the accelerator, we determine the error threshold. When the errors lager than this threshold occurs, the re-computation (rollback recovery) are performed for obtaining exact outputs. The threshold should be selected such that the inclusion of these exact outputs in the total number of outputs leads to a final average quality meeting the user-defined average quality. For this purpose, similar to what has been suggested in [9], the errors, which are resulted from applying the training datasets to the synthesized circuit, are sorted in descending order. Then, in each step, we eliminate one of the errors and evaluate the resulting final average quality. When the desired quality is achieved, the loop is ended. The greatest error which is remained on the list is the appropriate

0278-0070 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCAD.2018.2846625, IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems

TABLE I BENCHMARKS AND THEIR DETAILS

Benchmark	Domain	Train Data	Test Data	Application Error Metric	# of Arith. Units	Power of Exact Quality Configuration (mW)
Sobel	Image Processing	256×256 pixel image	256×256 pixel image	Mean Pixel Diff	13	6.6
FIR Filter (6-tap)	Signal Processing	2K random	2K random samples	Mean Relative Error	11	15
DCT (8-point)	Compression	256×256 pixel image	256×256 pixel image	Mean Pixel Diff	42	35.8
Forwardk2j	Robotics	5K random (x, y) angles	5K random (x, y) angles	Mean Relative Error	25	13.2
FFT (8-point)	Signal Processing	5K random	5K random samples	Mean Relative Error	40	41.1

quality threshold. An equation similar to (9) is employed to update the average quality in each iteration.

Hybrid Mode: In the third mode, the user provides both the average quality and the error threshold constraints. In this mode, we use the iterative approach proposed in *Threshold* and *Average* modes. Similar to the *Average Mode*, the initial average quality for the accelerator is determined based on the user-defined average quality. In each iteration, first, to train the error predictors, the provided error threshold is used. If the average quality constraint is not met, we employ the technique described in the *Average Mode* to find a proper threshold (which is definitely lower than the provided one).

IV. EVALUATION AND DISCUSSION

A. Simulation Setup

In this part, the details of the setup considered for evaluating the proposed design framework is described.

Approximate Arithmetic Units: In this study, the reconfigurable adder and the multiplier proposed in [6] and [7], respectively, were used for the arithmetic unit library. Each of these units has four approximate levels (including exact level). To support negative computations required for our evaluation of the benchmarks, slight modifications on these units were performed. These arithmetic units were implemented with Verilog description language and synthesized by Synopsys Design Compiler using NanGate 45nm technology [29].

Light-weight Error Predictors: Exact carry-lookahead adders and exact Dadda multipliers were employed to design the light-weight error predictors. Similar to the accelerator, the predictors have been synthesized by Synopsys Design Compiler using the same library. Furthermore, we employed scikit-learn [30] to train them.

Benchmarks: Several benchmarks from different application domains including signal processing and robotics were considered in this work. Table I presents the benchmark domain, number of nodes in their DFG, train and test data sets, quality evaluation metric, and power consumption when running using exact units. In the cases of the FIR, Forwardk2j, and FFT benchmarks (Sobel and DCT), for finding accelerator configuration and training as well as testing the predictor, 32-bit uniform random datasets (images provided in AxBench [31]) were employed.

Energy Modeling: We have employed Synopsys Power Compiler to measure the energy consumption of the designed accelerator and its corresponding error predictor. The coefficients used by the error predictors were stored in memory tables (Subsection 3.C) where CACTI 6.5 [32]was exploited to measure the energy consumption of accessing the tables.

B. Results

In the following subsections, first, the ROC curve for one of the benchmarks is shown and the influence of SMOTE in improving the classification accuracy is studied. Then, we demonstrate how our proposed quality management framework controls the output errors caused by the approximation in each quality management mode. Also, the proposed HLS (scheduling and binding) algorithm is assessed by comparing its performance with that of the algorithm proposed in [8]. For this study, the output errors were measured using the error metrics stated in Table I and normalized with respect to the case where all the operations were approximated using the lowest quality functional units for each application. Additionally, the output quality corresponds to the output error. For instance, an error threshold of 70% corresponds to a quality threshold of 30%. All the quality evaluations are performed using simulations. However, one may use analytical approaches (see, *e.g.*, [33] and [8]) which are faster at the cost of lower accuracy (about 10% inaccuracy in the approach presented in [8]).

1) Error Predictors and the Influence of SMOTE

As discussed in Subsection 3.C, we employed ROC plots to compare the classifiers for choosing the one more appropriate for our class of interest. In addition, to improve the accuracy of the classifiers, the SMOTE algorithm was utilized. To illustrate the effect of SMOTE, let us start with the study of the ROC plot for the *FIR Filter* benchmark given in Fig. 6. In this case, the quality threshold and the average quality of the accelerator are determined such that the final average quality would be 90%. As mentioned in Subsection 3.C.2, the power constraint of the accelerator limits the number of the basic classifiers which can be employed. In this benchmark (FIR Filter), this limitation is two. Therefore, we have four choices, namely one tree, two trees, one linear predictor, and two linear predictors, which are represented in ROC space with eight points in the figure. Half of the points which have solid markers correspond to the four chosen classifiers trained with the original training data, and the other half (hollow markers) illustrates these classifiers when they are trained after applying the SMOTE algorithm to the original training data. As it is obvious from the figure, using two basic classifiers (compared to using one) increases the true positive rate of the classifiers. Also, the figure indicates that the application of the SMOTE algorithm, improves the performance of the classifiers. Additionally, the figure indicates that the two classifiers lie on the convex hull where (8) was utilized to choose between these two candidates. Considering the fact that the qualities lower than the quality threshold must be avoided, a large number (cost) was assigned as the

0278-0070 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCAD.2018.2846625, IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems

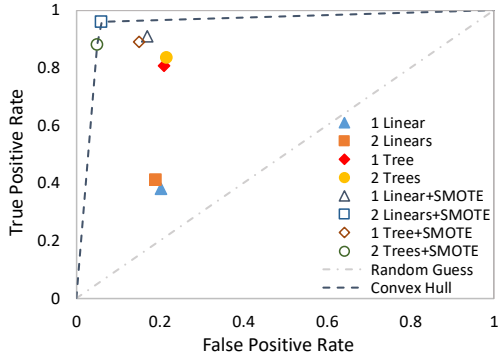


Fig. 6. ROC plot for FIR Filter benchmark at 90% target output quality.

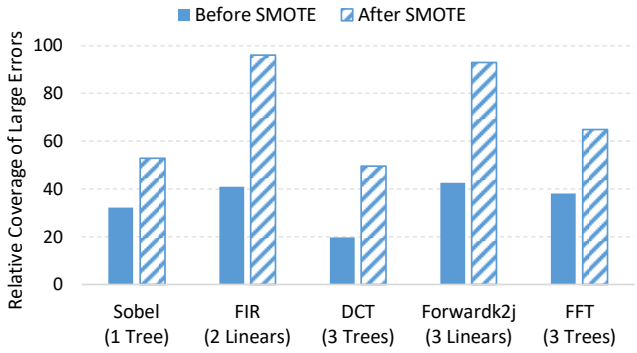


Fig. 7. Relative coverage of large errors at 90% target output quality.

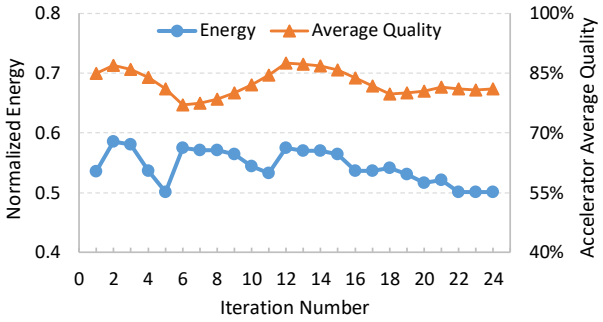


Fig. 8. Searching for the accelerator average quality with the minimum energy consumption in Threshold Mode (DCT benchmark at 50% target quality threshold).

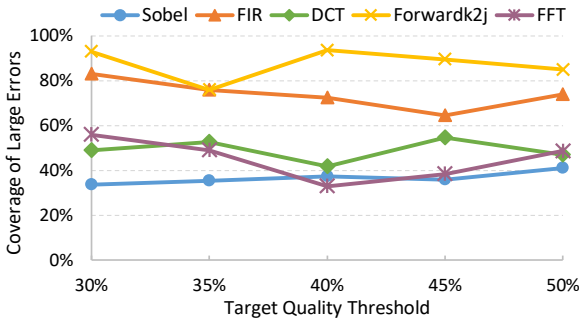


Fig. 9. Relative coverage of large errors by the considered error predictors under different expected quality thresholds in Threshold Mode.

misclassification cost of the positive class. Hence, we choose a classifier which has a larger TPR value. In this benchmark, the classifier comprised of two basic linear classifiers had the largest TPR value and was selected.

Finally, the chosen classifier and the coverage percentages of large errors at the target quality of 90% for all the benchmarks are shown in Fig. 7. It is worth noting that our simulation study indicates that, for each benchmark, the predictor configuration should be determined based on the output error distribution independent from the chosen type of the quality management and the quality constraints. The predictor configurations employed for the simulations performed in this article are shown in Fig. 7.As the results show, in all the benchmarks, the coverage increased by applying the SMOTE to the training data. In the studied benchmarks, the largest coverage increase (~55%) belonged to *FIR Filter* benchmark.

2) Threshold Mode Evaluation

In this mode, as elaborated in Subsection 3.D, all the output elements should have qualities higher than a given quality threshold. The challenge is to find the best value for the accelerator average quality in terms of the total energy consumption. Fig. 8 shows the results of the iterative procedure for obtaining the accelerator average quality (see Subsection 3.D) and normalized energy for the *DCT* benchmark versus the iteration number. As is observed from the figure, the best average quality is found after 24 iterations. In our simulations, we found that among the processes performed in each iteration, the quality determination phase had a larger runtime (*e.g.*, 80% of the total time for the DCT benchmark). It is worth noting that for DCT benchmark, which has the largest DFG among our considered benchmarks, one iteration of the whole framework (at the offline stage) takes 16 minutes on a system with Core i7 6700HQ as CPU and 8 GBs of RAM. The feature selection step is not included in this runtime. Note that the maximum runtime for the feature selection was 15s which belonged to Sobel benchmark.

Let us define *relative coverage* as the ratio of the detected large errors to the total elements with qualities below the given threshold (see, *e.g.*, [1]). The ideal value for the relative coverage is 100% which means all the large errors are detected soundly. Fig. 9 shows the relative coverage for a range of quality thresholds for different benchmarks. The best (worst) achieved value by our proposed design flow was 93% (33%) in the case of the *Forwardk2j* (*FFT*) benchmark. Additionally, Fig. 10 shows the ratio of the rollbacks for each benchmark. It plots both the number of the rollbacks in the ideal predictor case (denoted by "Oracle" in the figure) and the Achilles framework. Due to the false positive prediction of the employed predictor in the case of Achilles, the number of rollbacks is more than the ideal case. As the figure demonstrates, the amount of Oracle rollbacks are lower than 5% of the total invocations. Using the Achilles in Sobel benchmark for instance, about 40% of these 5% are detected. In other words, the quality constraint (the maximum of error) is violated for about 3% of the outputs due to the false negatives. However, the predictors perform better in the cases of FIR, Forwardk2j, and FFT compared to the cases of Sobel and DCT. Other approaches, such as PCA and LDA [20], may be employed to increase the accuracy of the prediction at the cost of imposing some energy overheads.

The normalized energy consumption versus the target average quality for the synthesized designs when the proposed

0278-0070 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCAD.2018.2846625, IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems

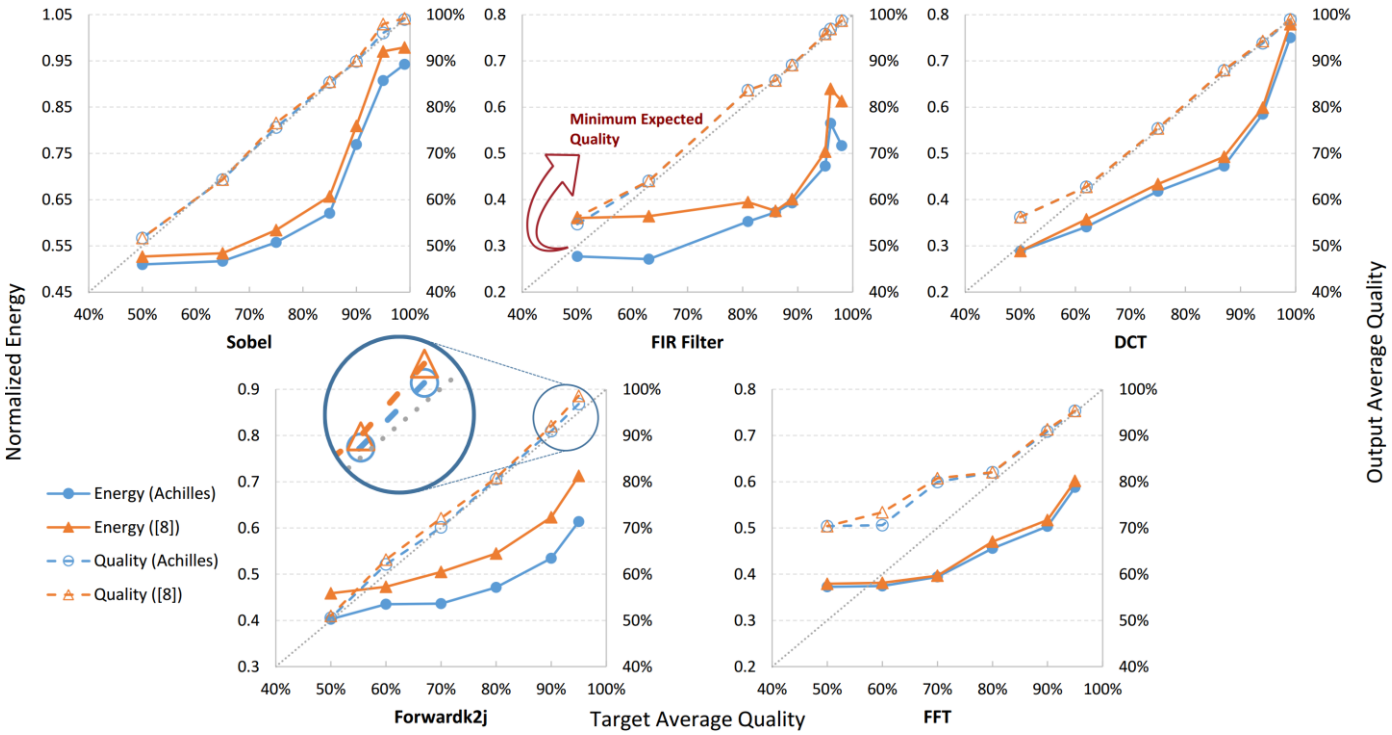


Fig. 12. Energy consumption and output average quality of all the benchmarks under different expected average output quality levels in Average Mode.

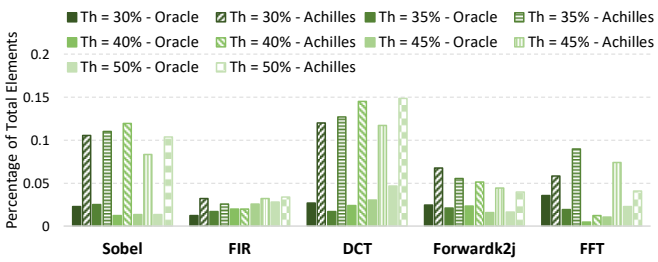


Fig. 10. The percentage of total elements required to be re-executed under different expected quality thresholds in the case of Threshold Mode.

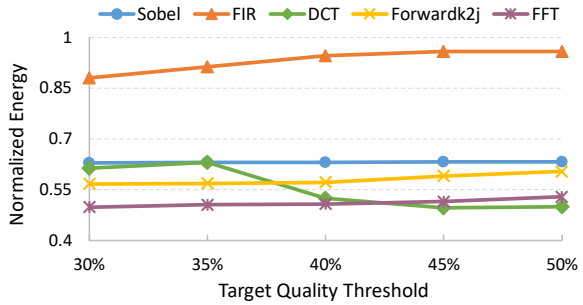


Fig. 11. Normalized energy consumption of Achilles under different expected quality thresholds in the Threshold Mode.

TABLE II
ENERGY CONSUMPTIONS FOR DCT BENCHMARK IN THE THRESHOLD MODE

Target Quality Threshold	30%	35%	40%	45%	50%
Achilles Energy (nJ)	295.3	303.4	308	315.9	326.7
Accelerator Part (nJ)	116.3	130.8	122.5	149.3	138.9
Rollback Part (nJ)	92.6	86.2	99.1	80.2	101.4
Predictor Part (nJ)	86.4	86.4	86.4	86.4	86.4
Accelerator Energy (nJ)	480.7	481	587.3	636.3	652.7

design flow was exploited is shown in Fig. 11. The energy consumption for each benchmark includes the energy consumed by the accelerator in both approximate and exact (rollback) operations, and by the error predictors. In addition, the energy consumptions were normalized to the energy of an accelerator configured in such a way that it did not violate the given quality constraint without using the error predictors. As the results indicate, for all the studied cases, the circuit designed by Achilles consumed less energy than the plain accelerator, which is not equipped with an error predictor. The energy gains, however, were different at different quality thresholds. For instance, in the case of the DCT benchmark, more gains were achieved at higher thresholds, while for the FIR Filter benchmark, the energy gains were reduced when the given quality threshold was increased. For the Sobel benchmark, the normalized energy was not changed considerably for the given target quality thresholds. In other words, although the total energy consumption increased by enlarging the target quality threshold, the energy consumption of the plain accelerator increased too causing the normalized energy to change differently. For being more explicit, the energy consumption values for the DCT benchmark are given in Table II. It can be seen that the total energy consumption in both cases (Achilles and only the accelerator) increases when the desired quality threshold increases. However, as is evident from Fig. 11, the ratio does not increase monotonically. It also demonstrates that although the accelerator and rollback parts consume non-negligible energy in the case of Achilles, the total energy consumption is considerably less than using a plain approximate accelerator.

3) *Average Mode Evaluation*

In the *Average Mode*, the user provides the average of the quality as the constraint. For this mode, since the *Approximation-Aware HLS* [8] algorithm proposed in [8] also

0278-0070 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCAD.2018.2846625, IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems

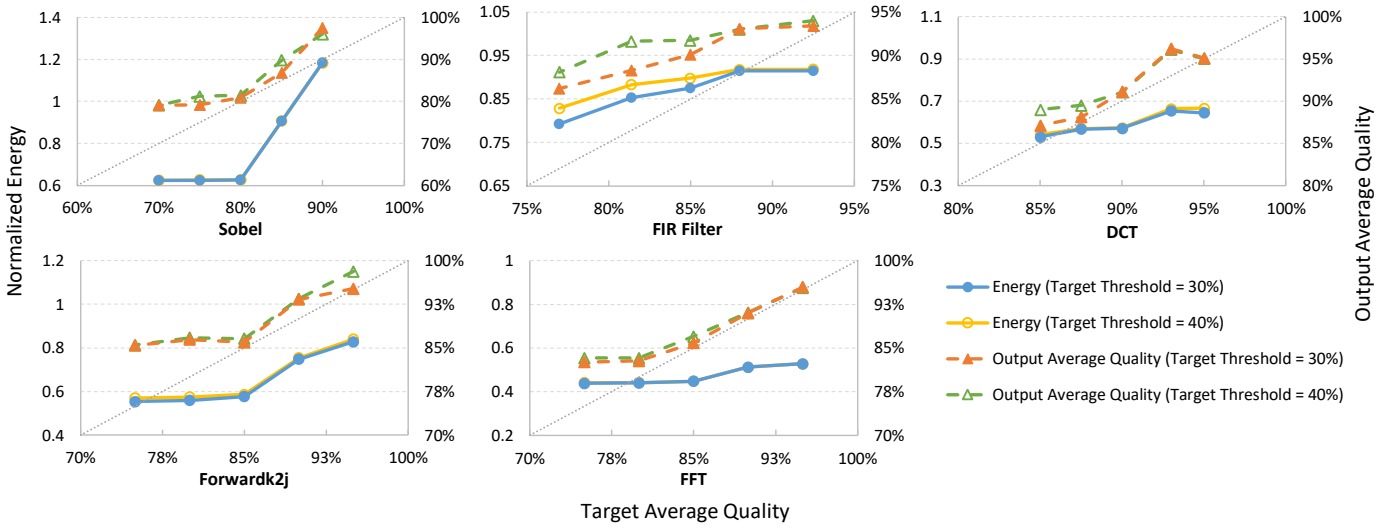


Fig. 13. Energy consumption and average output quality of the benchmarks under different expected average output quality levels in Hybrid Mode.

considered the average quality constraint, we compare the effectiveness of our proposed HLS algorithm with that of this work. All the energy consumptions were normalized with respect to the case when the accelerator was designed to run the application in the exact mode (100% quality). As the results shown in Fig. 12 indicate, for all the benchmarks, Achilles saved energy up to 10% more than the *Approximation-Aware HLS* algorithm. Also, neither of the works violated the quality constraint. The minimum average quality is also shown in Fig. 12. Since in our proposed technique, we maximize energy reduction by pushing the approximate computing to the point that the minimum quality requirement is satisfied, our quality values are closer to the minimum target quality. In addition, the use of predictors in Achilles, enables the use of more aggressive approximation in the accelerator for more energy consumption reduction.

4) Hybrid Mode Evaluation

As mentioned before, both the average quality and the quality threshold are provided in this mode. The normalized energy consumptions for different target average qualities as well as the output average qualities for two different quality thresholds are depicted in Fig. 13. The energy consumptions were normalized the same way as that of the threshold mode. When the target average quality increases and target quality threshold is fixed, the energy gains obtained from the approximation decrease. The rate of the reduction in the energy gain depends on each application and its configuration. For instance, in the *Sobel* benchmark, although the energy change trend is positive, the energy gain decreases abruptly for high target qualities. In most cases, however, employing error predictors reduced the total energy consumption (up to 57% for the *FFT* benchmark). Also, as is obvious from the figure, the average quality constraints are always met for the cases. Considering the fact that relative coverage depends on the benchmark (not the target quality), its results are similar to *Threshold Mode* and are not included to avoid tautology.

V. Conclusion

In this work, we proposed an accuracy-aware design framework which synthesizes the high-level description of the input application with the objective of minimizing the energy consumption. The framework, which was called Achilles, synthesized the kernel of the input application. The quality of the synthesized design was enhanced by utilizing light-weight error predictors based on machine learning classifiers. In the proposed design flow, to reach a higher quality with the smaller energy consumption, after performing the high-level synthesis (HLS), an exploration was conducted to select the best error predictor for the design. To synthesize the datapath efficiently, the proposed HLS algorithm made use of accuracy configurable arithmetic units in the given library. Experiments demonstrate that proposed framework gain significant benefit. Our application suite used less energy consumption up to 51%, 72%, and 57% in Threshold, Average, and Hybrid modes respectively. In addition, in order to enhance the classification accuracy of predictors, SMOTE method was employed, which increased relative coverage of large errors up to 55%.

References

[1] D. S. Khudia, B. Zamirai, M. Samadi, and S. Mahlke, “Rumba: An online quality management system for approximate computing,” in *International Symposium on Computer Architecture (ISCA)*, 2015, pp. 554–566.

[2] D. Mahajan, A. Yazdanbaksh, J. Park, B. Thwaites, and H. Esmaeilzadeh, “Towards Statistical Guarantees in Controlling Quality Tradeoffs for Approximate Acceleration,” in *International Symposium on Computer Architecture (ISCA)*, 2016, pp. 66–77.

[3] S. Narayanamoorthy, H. A. Moghaddam, Z. Liu, T. Park, and N. S. Kim, “Energy-Efficient Approximate Multiplication for Digital Signal Processing and Classification Applications,” *IEEE Trans. Very Large Scale Integr. Syst.*, vol. 23, no. 6, pp. 1180–1184, 2015.

[4] A. Momeni, J. Han, P. Montuschi, and F. Lombardi, “Design and Analysis of Approximate Compressors for Multiplication,” *IEEE Trans. Comput.*, vol. 64, no. 4, pp. 984–994, 2015.

[5] M. Shafique, W. Ahmad, R. Hafiz, and J. Henkel, “A Low Latency Generic Accuracy Configurable Adder,” in *Design Automation Conference (DAC)*, 2015, p. 86:1–86:6.

[6] O. Akbari, M. Kamal, A. Afzali-Kusha, and M. Pedram, “RAP-CLA: A Reconfigurable Approximate Carry Look-Ahead Adder,” *IEEE Transactions on Circuits and Systems II: Express Briefs*, 2017.

[7] O. Akbari, M. Kamal, A. Afzali-Kusha, and M. Pedram, “Dual-Quality 4:2 Compressors for Utilizing in Dynamic Accuracy Configurable Multipliers,” *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 25, no. 4, pp. 1352-1361, 2017.

[8] C. Li, W. Luo, S. S. Sapatnekar, and J. Hu, “Joint Precision Optimization and High Level Synthesis for Approximate Computing,” in *Design Automation Conference (DAC)*, 2015, p. 104:1–104:6.

0278-0070 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.

This article has been accepted for publication in a future issue of this journal, but has not been fully edited. Content may change prior to final publication. Citation information: DOI 10.1109/TCAD.2018.2846625, IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems

[9] T. Wang, Q. Zhang, N. S. Kim, and Q. Xu, "On Effective and Efficient Quality Management for Approximate Computing," in *International Symposium on Low Power Electronics and Design (ISLPED)*, 2016, pp. 156–161.

[10] V. K. Chippa, D. Mohapatra, K. Roy, S. T. Chakradhar, and A. Raghunathan, "Scalable Effort Hardware Design," *IEEE Trans. Very Large Scale Integr. Syst.*, vol. 22, no. 9, pp. 2004–2016, 2014.

[11] A. Raha, S. Venkataramani, V. Raghunathan, and A. Raghunathan, "Quality Configurable Reduce-and-rank for Energy Efficient Approximate Computing," in *Design, Automation & Test in Europe (DATE)*, 2015, pp. 665–670.

[12] Q. Zhang, T. Wang, Y. Tian, F. Yuan, and Q. Xu, "ApproxANN: An approximate computing framework for artificial neural network," in *Design, Automation & Test in Europe (DATE)*, 2015, pp. 701–706.

[13] J. Miao, A. Gerstlauer, and M. Orshansky, "Approximate logic synthesis under general error magnitude and frequency constraints," in *International Conference on Computer-Aided Design (ICCAD)*, 2013, pp. 779–786.

[14] S. Venkataramani, A. Sabne, V. Kozhikkottu, K. Roy, and A. Raghunathan, "SALSA: Systematic logic synthesis of approximate circuits," in *Design Automation Conference (DAC)*, 2012, pp. 796–801.

[15] A. Ranjan, A. Raha, S. Venkataramani, K. Roy, and A. Raghunathan, "ASLAN: Synthesis of Approximate Sequential Circuits," in *Design, Automation & Test in Europe (DATE)*, 2014, p. 364:1–364:6.

[16] K. Nepal, Y. Li, R. I. Bahar, and S. Reda, "ABACUS: A Technique for Automated Behavioral Synthesis of Approximate Computing Circuits," in *Design, Automation & Test in Europe (DATE)*, 2014, p. 361:1–361:6.

[17] W. Baek and T. M. Chilimbi, "Green: A Framework for Supporting Energy-conscious Programming Using Controlled Approximation," in *ACM SIGPLAN Conference on Programming Language Design and Implementation*, 2010, pp. 198–209.

[18] M. Samadi, J. Lee, D. A. Jamshidi, A. Hormati, and S. Mahlke, "SAGE: Self-tuning Approximation for Graphics Engines," in *International Symposium on Microarchitecture*, 2013, pp. 13–24.

[19] F. Provost and T. Fawcett, "Analysis and Visualization of Classifier Performance: Comparison Under Imprecise Class and Cost Distributions," in *International Conference on Knowledge Discovery and Data Mining*, 1997, pp. 43–48.

[20] S. Theodoridis and K. Koutroumbas, *Pattern recognition*. Academic Press, 2009.

[21] N. V Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic Minority Over-sampling Technique," *J. Artif. Int. Res.*, vol. 16, no. 1, pp. 321–357, 2002.

[22] T. G. Dietterich, "An Experimental Comparison of Three Methods for Constructing Ensembles of Decision Trees: Bagging, Boosting, and Randomization," *Mach. Learn.*, vol. 40, no. 2, pp. 139–157, 2000.

[23] R. E. Schapire, "The strength of weak learnability," *Mach. Learn.*, vol. 5, no. 2, pp. 197–227, 1990.

[24] J. Han, M. Kamber, J. Pei, J. Han, M. Kamber, and J. Pei, "8 – Classification: Basic Concepts," in *Data Mining*, 2012, pp. 327–391.

[25] R. Kohavi, "A Study of Cross-validation and Bootstrap for Accuracy Estimation and Model Selection," in *International Joint Conference on Artificial Intelligence - Volume 2*, 1995, pp. 1137–1143.

[26] J.-H. Kim, "Estimating Classification Error Rate: Repeated Cross-validation, Repeated Hold-out and Bootstrap," *Comput. Stat. Data Anal.*, vol. 53, no. 11, pp. 3735–3745, 2009.

[27] A. M. Molinaro, R. Simon, and R. M. Pfeiffer, "Prediction Error Estimation: A Comparison of Resampling Methods," *Bioinformatics*, vol. 21, no. 15, pp. 3301–3307, 2005.

[28] W. T. J. Chan, A. B. Kahng, S. Kang, R. Kumar, and J. Sartori, "Statistical analysis and modeling for error composition in approximate computation circuits," in *International Conference on Computer Design (ICCD)*, 2013.

[29] "NanGate FreePDK45 Open Cell Library," http://www.nangate.com/?page_id=2325, accessed: 2017-11-9.

[30] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine Learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[31] A. Yazdanbakhsh, D. Mahajan, H. Esmaeilzadeh and P. Lotfi-Kamran, "AxBench: A Multiplatform Benchmark Suite for Approximate Computing," *IEEE Design & Test*, vol. 34, no. 2, pp. 60-68, 2017.

[32] N. P. J. Naveen Muralimanohar Rajeev Balasubramonian, "CACTI 6.0: A Tool to Model Large Caches," *International Symposium on Microarchitecture*. 2007.

[33] S. Mazahir, O. Hasan, R. Hafiz, M. Shafique, and J. Henkel, "Probabilistic Error Modeling for Approximate Adders," *IEEE Transactions on Computers*, vol. 66, no. 3, pp. 515–530, 2017.



Shayan Tabatabaei-Nikkhah received the M.Sc. degree in electrical engineering from the University of Tehran, Tehran, Iran in 2018. His research interests include approximate computing, electronic design automation, low-power design, hardware/software co-design, and computer architecture.



Mahdi Zahedi received the M.Sc. degree in electrical engineering from the University of Tehran, Tehran, Iran, in 2017. He is currently pursuing the Ph.D. degree in Electrical Engineering with the Eindhoven University of Technology, Eindhoven, the Netherlands. His main areas of research interest are reconfigurable design, and system-level HW/SW co-design.



Mehdi Kamal received the Ph.D. degree from the University of Tehran, Tehran, in 2013, in computer engineering. He is currently the assistant professor with the School of Electrical and Computer Engineering of the University of Tehran, Iran. His current research interests include reliability in nanoscale design, approximate computing, neuromorphic computing, and low-power design.



Ali Afzali-Kusha received the Ph.D. degree from the University of Michigan, Ann Arbor, MI, USA, in 1994, in electrical engineering. He has been with the University of Tehran, since 1995, where he is currently a Professor of the School of Electrical and Computer Engineering. His current research interests include low-power high-performance design methodologies from the physical design level to the system level for nanoelectronics era.



Massoud Pedram (F'01) received the Ph.D. degree in electrical engineering and computer sciences from the University of California at Berkeley, Berkeley, CA, USA, in 1991. In 1991, he joined the Ming Hsieh Department of Electrical Engineering, University of Southern California (USC), Los Angeles, CA, USA, where he currently is the Stephen and Etta Varra Professor with the USC Viterbi School of Engineering. His current research interests include energy-efficient computing, system-level power management and power-aware design methodologies, and low power circuit design techniques.

0278-0070 (c) 2018 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission. See http://www.ieee.org/publications_standards/publications/rights/index.html for more information.